
Orientación
a Objetos
Platzi
Server A
Server B
Server C
Autoplay

Contenido

¿Porque aprender POO? 4

¿Que resuelve la Programación Orientada a Objetos? 5

Orientación a Objetos 5

Lenguajes Orientados a Objetos 6

Diagramas de Modelado OMT, UML 7

¿Qué es UML? 8

Asociación 9

Herencia 9

Agregación 10

Composición 10

No entiendo esto xd, compruebalo cuando ya entiendas el tema 11

Objetos 12

Abstracción de objetos 13

Abstracción y Clases 15

Modularidad 15

Clases en UML y su Sintaxis en código 16

¿Qué es la herencia? 17

Materiales para Guiarme 19

Objetos, método constructor y su sintaxis en código 22

22

Método Constructor 22

Investiga que es this, ArrayList, Map, super, Métodos constructores. 26

Estoy haciendo la practica ahorita sacamos apunte xd… 26
Aplicando herencia en código 27
Otros tipos de herencia (Payment) 28
Account 29
Encapsulamiento 30
Getter y Setters 30
Generando Polimorfismo en JAVA 31
Diagrama UML de Uber 31
Asociación 32
Herencia 33
Composición 35

¿Porque aprender POO?

La programación orientada a objetos (POO) o Object Oriented Programm (OOP) nos sirve para enfocarnos a:

- Programar mas rápido
- Dejar de ser programador Jr:

  Ya que los reclutadores piden conocimientos solidos de POO.
- Dejar de copiar y pegar código

Para continuar tenemos que entender estos conceptos e implementarlos en nuestro código

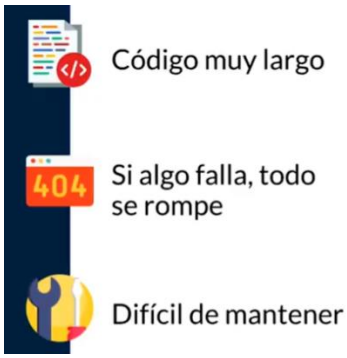


La programación orientada a objetos tiene cuatro características principales:
Encapsulamiento. Quiere decir que oculta datos mediante código.
Abstracción. Es como se pueden representar los objetos en modo de código.
Herencia. Es donde una clase nueva se crea a partir de una clase existente.
Polimorfismo. Se refiere a la propiedad por la que es posible enviar mensajes sintácticamente iguales a objetos de tipos distintos.

Para lograrlo es importante desarrollar estas habilidades:



¿Que resuelve la Programación Orientada a Objetos?



Tiene el objetivo de resolver los problemas que nos deja la programación estructurada.

De esta manera simplifica la programación evitando el código Espagueti

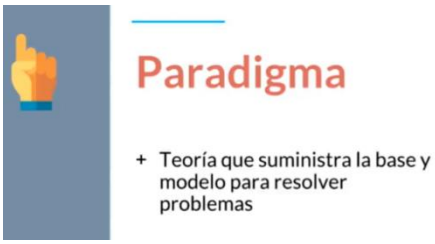


Orientación a Objetos





POO es un paradigma



POO se descompone en estos 4 elementos que en conjunto se convierten en objetos



Esto lo realiza atreves de los 4 pilares de POO



Lenguajes Orientados a Objetos



Lenguajes de Programación

- Java
- PHP
- Python
- JavaScript
- C#
- Ruby
- Kotlin

Java



- Orientado a Objetos naturalmente
- Android
- Server Side

`.java`

PHP



- Lenguaje interpretado
- Pensado para la Web

`.php`

Python



- Diseñado para ser fácil de usar
- Mútiples usos: Web, Server Side, Análisis de Datos, Machine Learning, etc.

`.py`

JavaScript



- Lenguaje interpretado
- Orientado a Objetos pero basado en prototipos
- Pensado para la Web

`.js`

Diagramas de Modelado OMT, UML

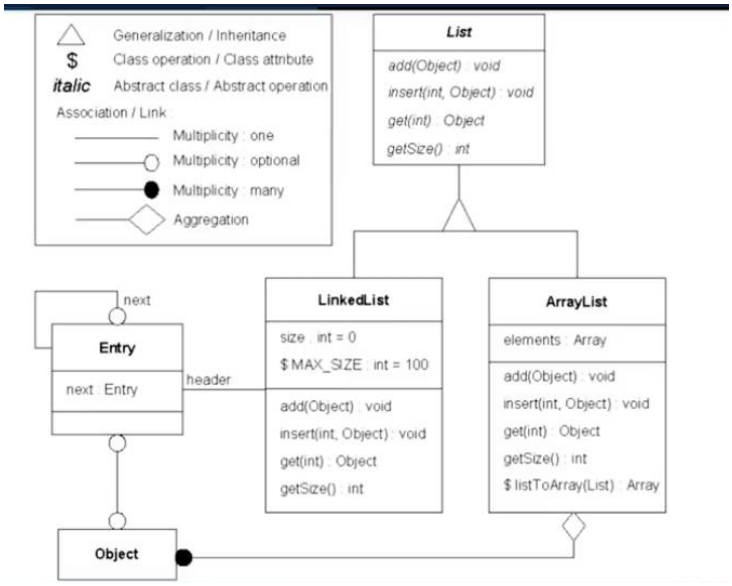


Actualmente OMT esta descontinuado, pero sirvió para crear UML que es lo básico y lo primero que piden en un proyecto

OMT
Object Modeling Techniques

OMT
Object Modeling
Techniques

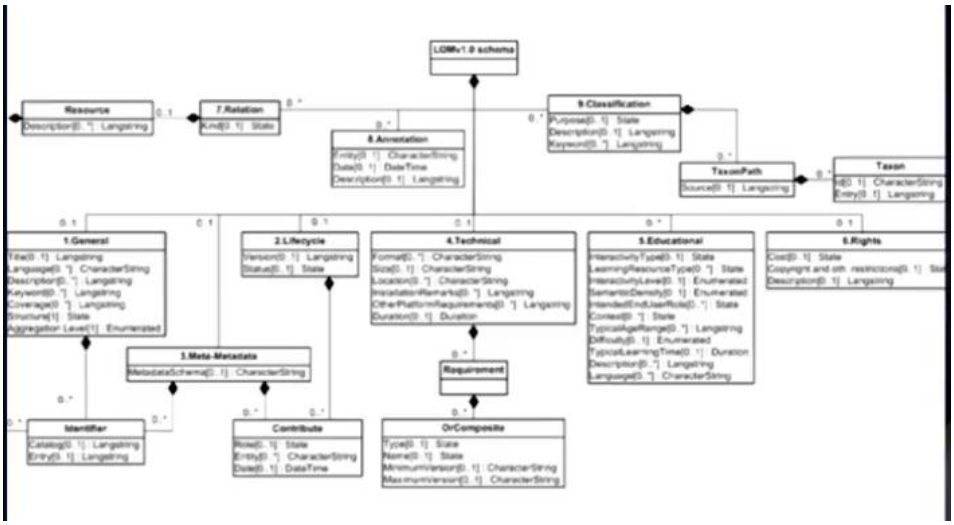
+ Metodología para el análisis orientado a objetos



UML
Unified Modeling Language
Lenguaje de Modelado Unificado

UML
Unified Modeling Language

+ Clases
+ Casos de Uso
+ Objetos
+ Actividades
+ Iteración
+ Estados
+ Implementación

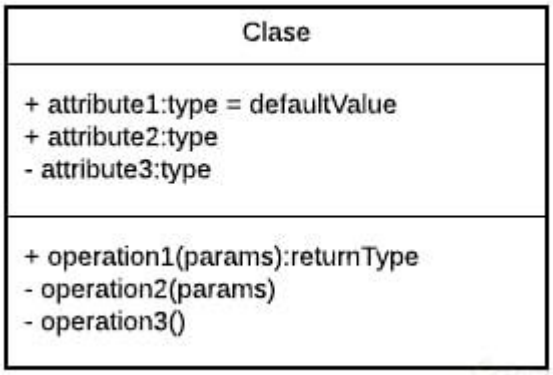


¿Qué es UML?

Como ya viste **UML** significa **Unified Modeling Language** el cual es un lenguaje estándar de modelado de sistemas orientados a objetos.

Esto significa que tendremos una **manera gráfica de representar una situación**, justo como hemos venido viendo. A continuación te voy a presentar los elementos que puedes utilizar para hacer estas representaciones.

Las **clases** se representan así:



En la parte superior se colocan los atributos o propiedades y debajo las operaciones de la clase. Notarás que el primer carácter con el que empiezan es un símbolo. Este denotará la visibilidad del atributo o método, esto es un término que tiene que ver con Encapsulamiento y veremos más adelante a detalle.

Estos son los niveles de **visibilidad** que puedes tener:

- **-** private
- **+** public
- **#** protected
- **~** default

Permite definir que partes del objeto son visibles (el interfaz publico), que partes son ocultas (privadas).

Miembros públicos: Describen que pueden hacer los objetos de esa clase.

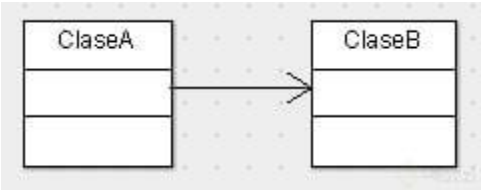
Miembros privados: describen la implementación de como lo hacen y pueden cambiar el valor de un atributo privado y los estados son privados y solo se pueden cambiar atraves del interfaz publico.

Una forma de representar las relaciones que tendrá un elemento con otro es a través de las flechas en UML, y aquí tenemos varios tipos, estos son los más comunes:

Asociación

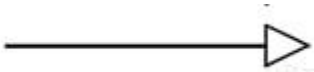


Como su nombre lo dice, notarás que cada vez que esté referenciada este tipo de flecha significará que ese elemento contiene al otro en su definición.
La flecha apuntará hacia la dependencia. Con esto vemos que la ClaseA está asociada y depende de la ClaseB.

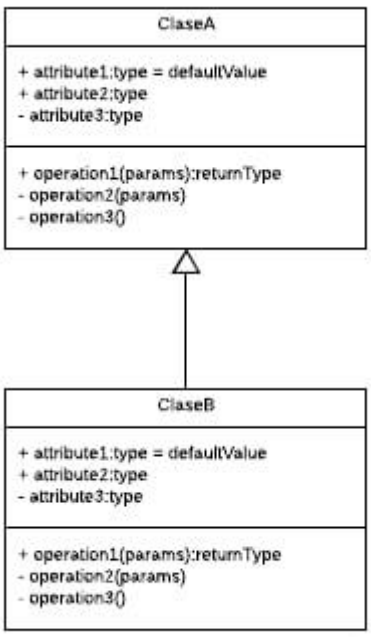


Herencia

Siempre que veamos este tipo de flecha se estará expresando la herencia.
La dirección de la flecha irá desde el hijo hasta el padre.



Con esto vemos que la ClaseB hereda de la ClaseA



Agregación

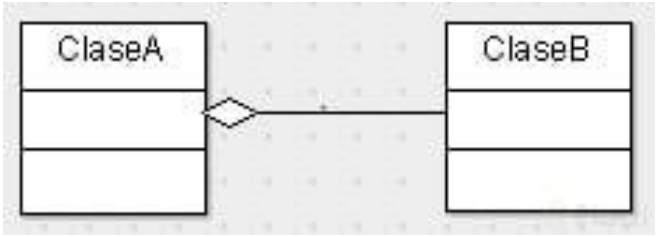
Este se parece a la asociación en que un elemento dependerá del otro, pero en este caso será:



Un elemento dependerá de muchos otros.

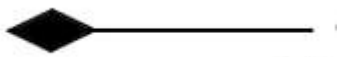
Aquí tomamos como referencia la multiplicidad del elemento. Lo que comúnmente conocerías en Bases de Datos como Relaciones uno a muchos.

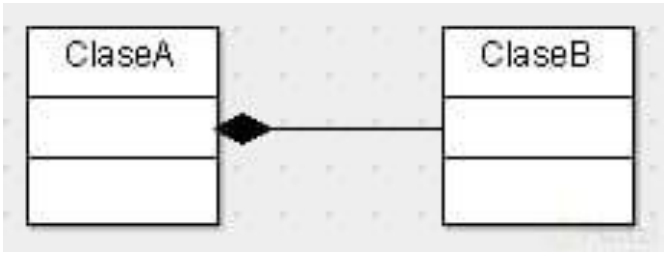
Con esto decimos que la ClaseA contiene varios elementos de la ClaseB. Estos últimos son comúnmente representados con listas o colecciones de datos.



Composición

Este es similar al anterior solo que su relación es totalmente compenetrada de tal modo que conceptualmente una de estas clases no podría vivir si no existiera la otra. Con esto terminamos nuestro primer módulo. Vamos al siguiente para entender cómo podemos hacer un análisis y utilizar estos elementos para construir nuestro diagrama de clases de Uber.



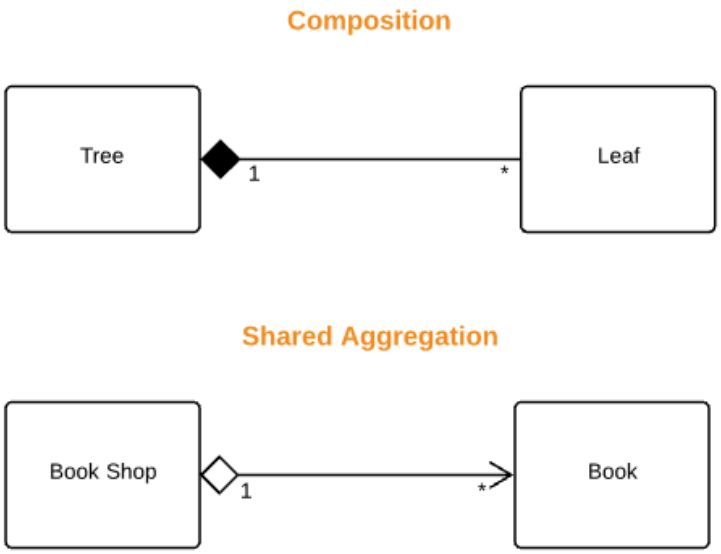


No entiendo esto xd, compruebalo cuando ya entiendas el tema

Alan Alberto Dromundo Arias
Hace un año

207

La **Agregación** y **Composición** no me habían quedado claros, así que busqué y encontré este diagrama que me resultó muy útil:



En la **composición**, el elemento *Tree* depende completamente del elemento *Leaf*, siendo el caso de que si el elemento *Leaf* desapareciera también lo haría el elemento *Tree*. Mientras que en la **agregación** si el elemento *Book* desaparece, o se vuelve 0, (esto me imagino que lo explicarán más adelante) el elemento *Book Shop* no lo haría.

Responder

Josue Granados
Hace un año

39

Gracias por los diagramas!
Solo que hay una pequeña **impresición** en el ejemplo de la relación entre la clase *tree* y la clase *leaf*.
En el ejemplo del diagrama la clase *leaf* no puede existir sin la clase *tree*. Yo he visto muchos arboles secos sin hojas, pero jamas he visto hojas flotando sin algún árbol que las sujete.
En las ***compositions*** siempre se apunta al padre de la relación y el hijo es que no puede existir sin el padre.
Te dejo un artículo muy bueno que habla sobre esto:
https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-aggregation-vs-composition/
Una vez mas, Gracias por los diagramas!

Objetos

- Cuando tengamos un problema lo primero que debemos hacer es **identificar Objetos**

Objetos

Los Objetos se refieren a una instancia en la memoria del sistema donde se ejecuta el programa. Tienen un estado interno que dependiendo del lenguaje, se pueden conocer los datos o variables de instancia y son aquellos



que tienen **propiedades** y **comportamientos**, también serán sustantivos.

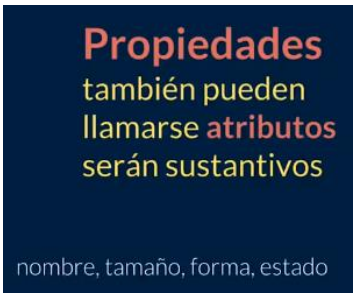
Los objetos son unidades independientes por lo que se pueden comunicar entre ellos mediante mensajes.

Pueden ser **Físicos** o **Conceptuales**





- Las **Propiedades** también pueden llamarse **atributos** y estos también serán sustantivos. Algunos atributos o propiedades son nombre, tamaño, forma, estado, etc. Son todas las características del objeto.



- Los **Comportamientos** serán todas las operaciones que el objeto puede hacer, suelen ser verbos o sustantivos y verbo. Algunos ejemplos pueden ser que el usuario pueda hacer login y logout.



Perro



Comportamientos

+ nombre
+ color
+ raza
+ altura

+ ladrar
+ comer
+ dormir
+ correr

Propiedades

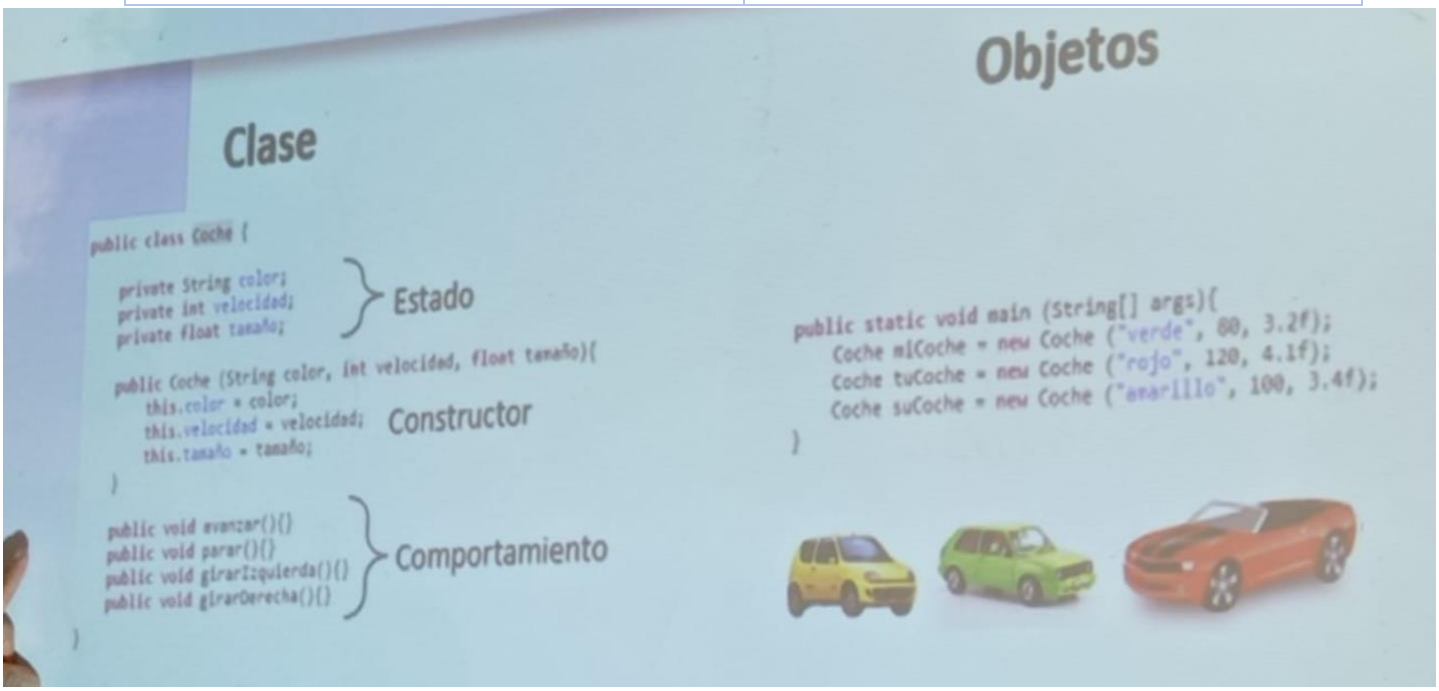
Abstracción de objetos

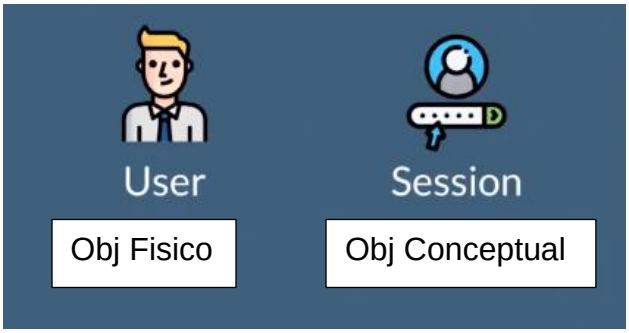


Abstracción Funcional	Abstracción de datos
Hay cosas que sabemos que los coches hacen pero no como lo hacen: ✓ Avanzar ✓ Parar ✓ Girar	Un coche puede tener ciertos atributos ✓ Color ✓ Velocidad ✓ Tamaño ✓ Peso ✓ Marca

A estas abstracciones se conocen como:

Comportamiento	**Estado**
Esta definido por sus métodos Métodos: contenedores de funciones	Esta representado por el contenido de sus atributos Atributos(variables):contenedores de valores





El Atributo no es el resultado, un ejemplo de esto sería si el usuario de llama Anahí, el atributo no es Anahí si no usuario, otro ejemplo seria llamarle al atributo verde cuando en realidad es color.

Para determinar sus atributos y comportamiento se debe de entender el contexto en que se encuentra el objeto



Los objetos interaccionan entre si enviando mensajes o delegar tareas. Esto lo logra mediante llamada a sus atributos o métodos y son tratados por la interfaz publica del objeto que los recibe.

El objeto receptor puede cambiar su estado, es decir “modificar sus atributos” o Enviando otros mensajes a otros atributos o métodos del mismo objeto (públicos o privados) o de otros objetos (públicos o accesibles desde ese objeto).

- ✓ **Una clase es una entidad abstracta**
- ✓ **Un objeto es una instancia de una clase**

►**Una clase se declara, un objeto se crea**

Abstracción y Clases



Una **Clase** es el modelo, plano o prototipo, por el cual nuestros objetos se van a construir y nos van a permitir generar más objetos.



Analizamos Objetos para crear Clases. Las Clases son los modelos sobres los cuales construiremos nuestros objetos.

Abstracción es cuando separamos los datos de un objeto para generar un molde. La abstracción es el proceso para crear una clase.

Modularidad

La modularidad va muy relacionada con las clases y es un principio de la Programación Orientado a Objetos y va de la mano con el Diseño Modular que significa dividir un sistema en partes pequeñas y estas serán nuestros módulos pudiendo funcionar de manera independiente.



La modularidad de nuestro código nos va a permitir

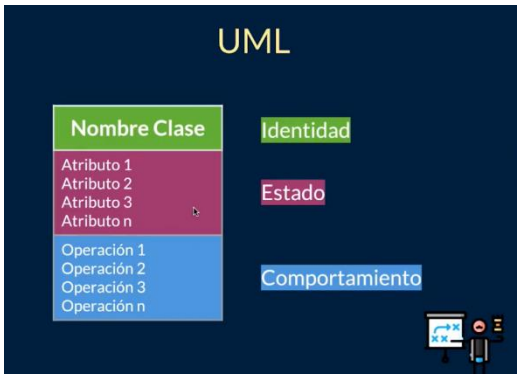
- Reutilizar
- Evitar colapsos
- Hacer nuestro código más mantenible
- Legibilidad
- Resolución rápida de problemas

Clase

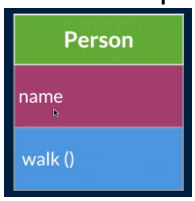
- Modularidad
- Divide el programa en diferentes partes o módulos / clases
- Separar las clases en archivos

Una buena práctica es separando las clases en archivos diferentes.

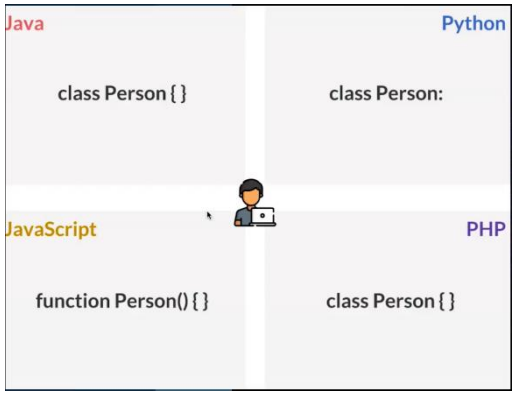
Clases en UML y su Sintaxis en código



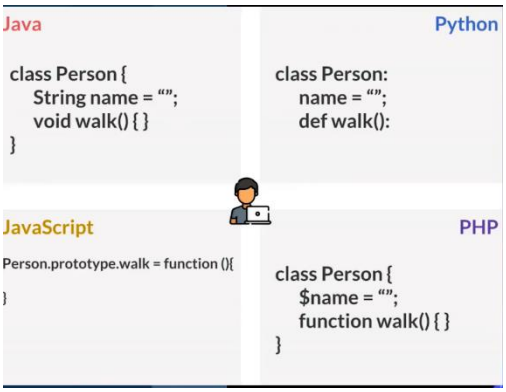
Estas son las partes de las clases representadas en UML… Un ejemplo pod ría ser una clase persona:



Para definir el objeto persona en código



Así se definen los atributos y metodos



¿Qué es la herencia?



DRY: Don’t Repeat Yourself

+Promueve la reducción de duplicación en programación

+Toda pieza de información nunca debería ser duplicada debido a que la duplicación incrementa la dificultad en los cambios y evolución.

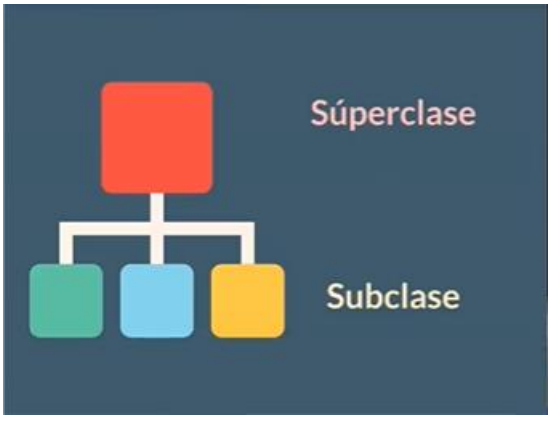
Para solucionar este problema tenemos que reutilizar Código.



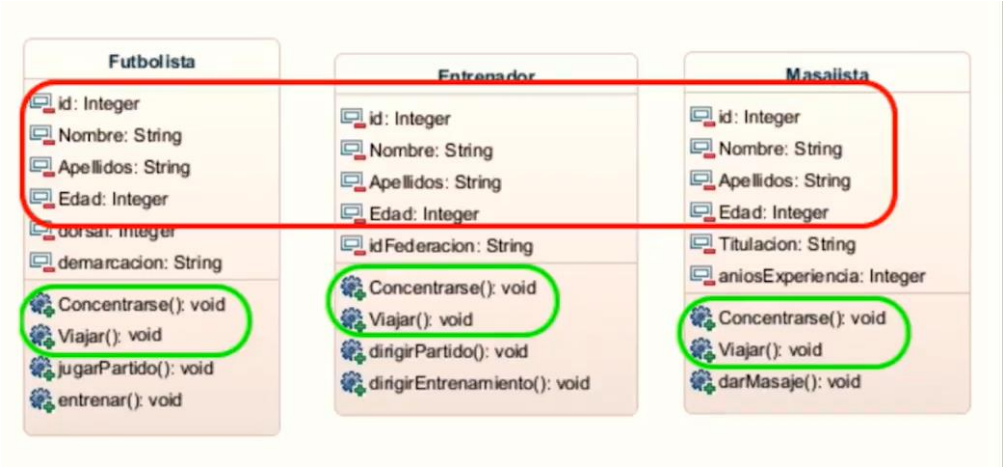


Lo que nos lleva a la HERENCIA, herramienta y pieza clave que nos permite reducir el codigo.

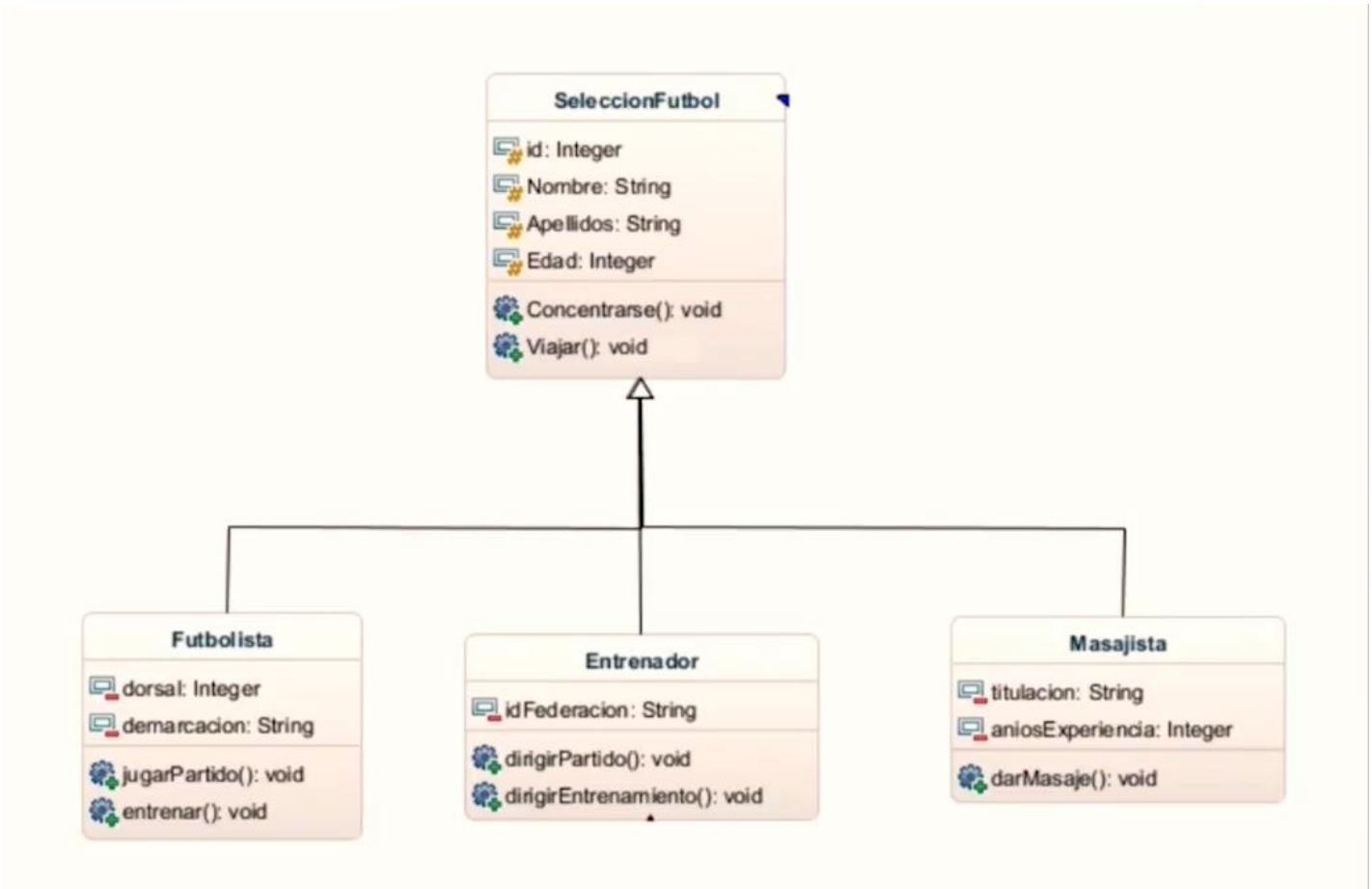




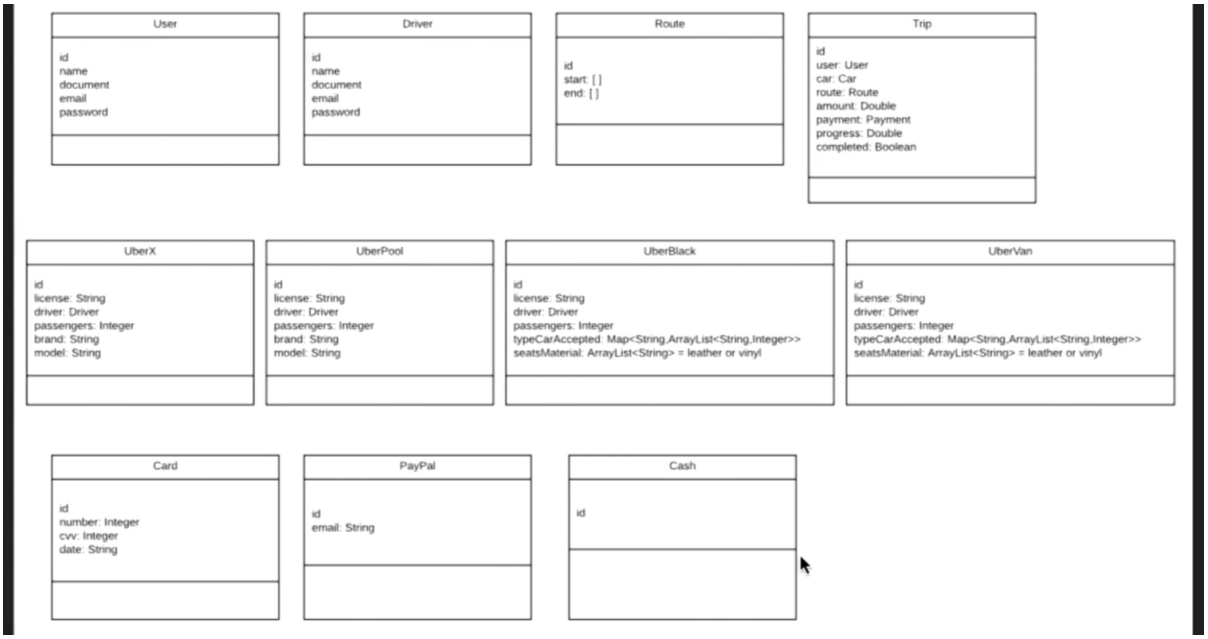
En cuanto detectamos que existen elementos repetidos, generamos una abstracción y crear una clase que tenga todos estos elementos en común

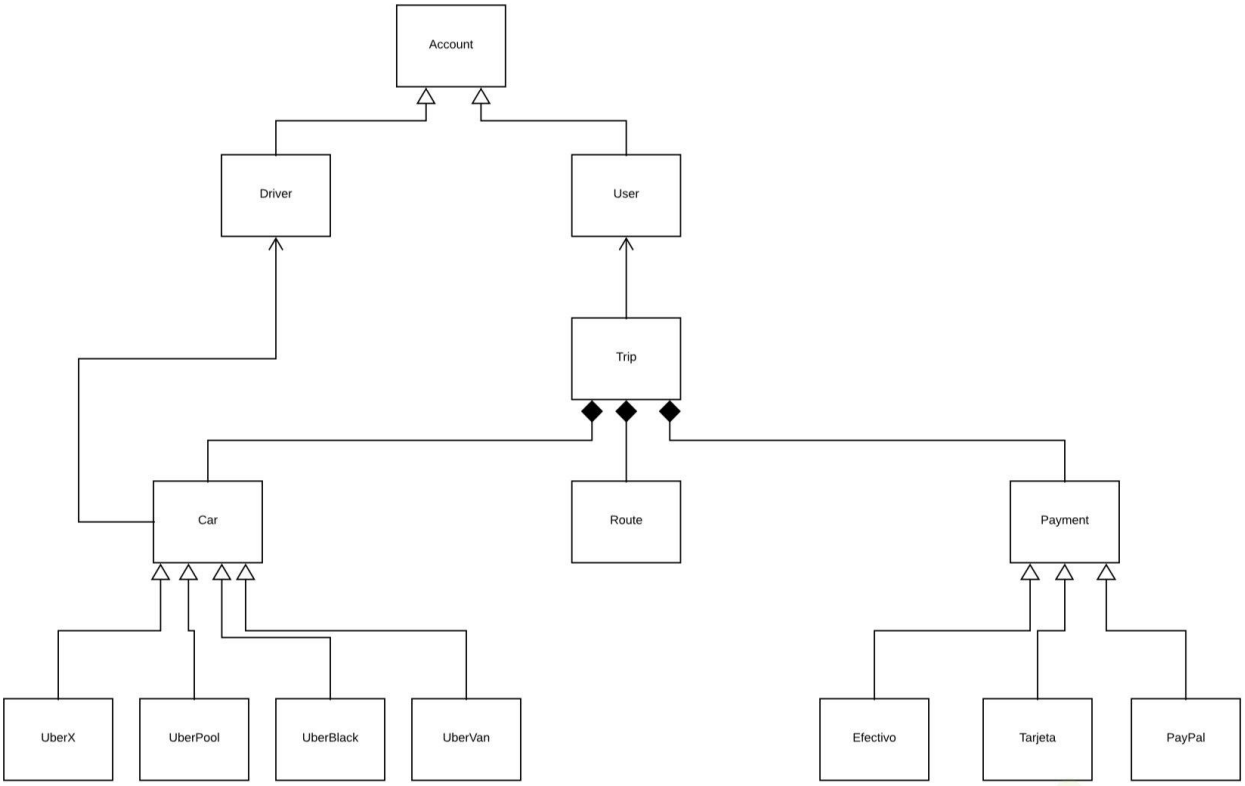


Todos los atributos y métodos que tiene la clase la clase padre o super clase serán heredados a las clases hijas o subclases

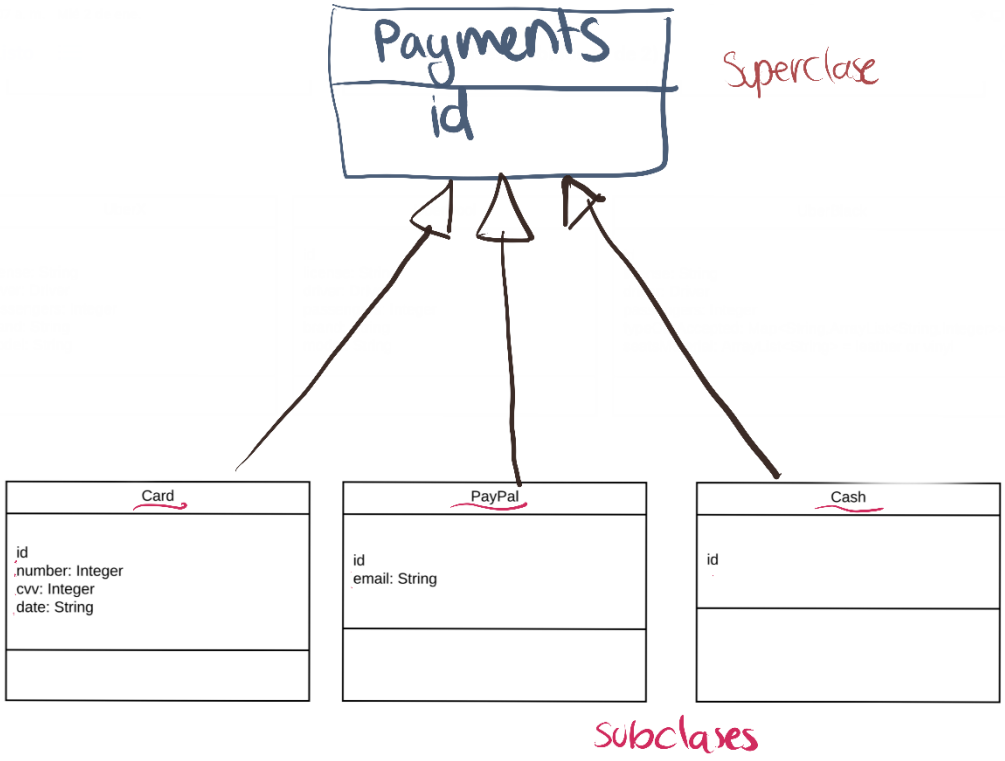


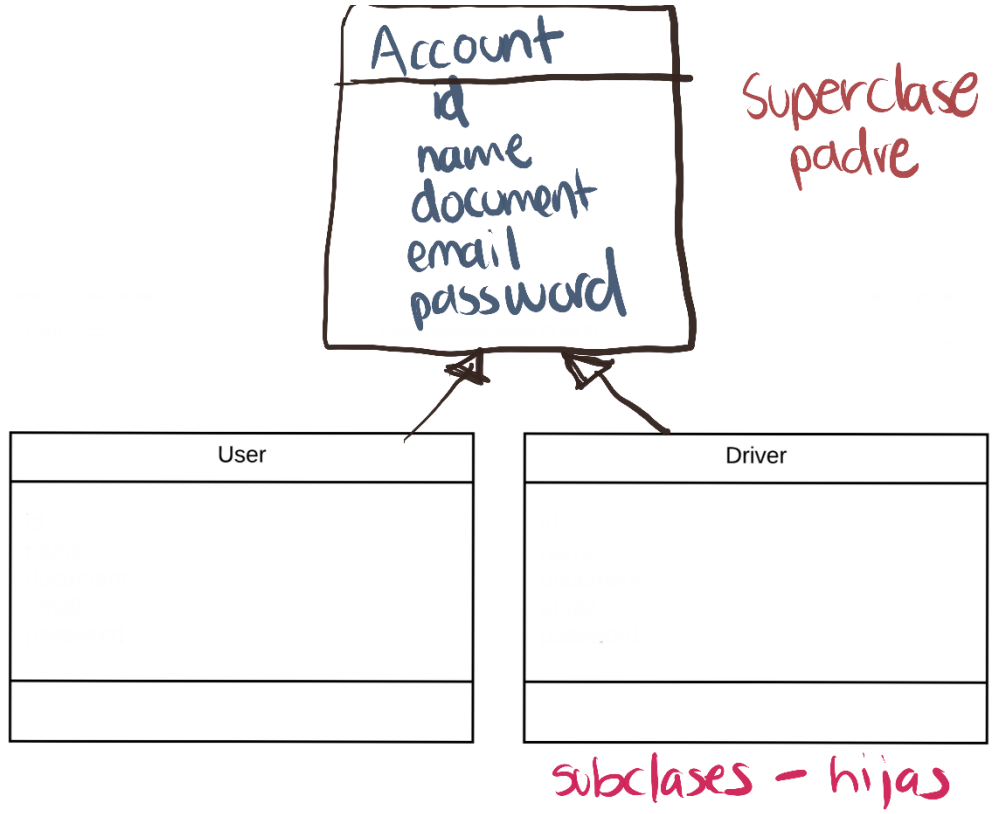
Materiales para Guiarme

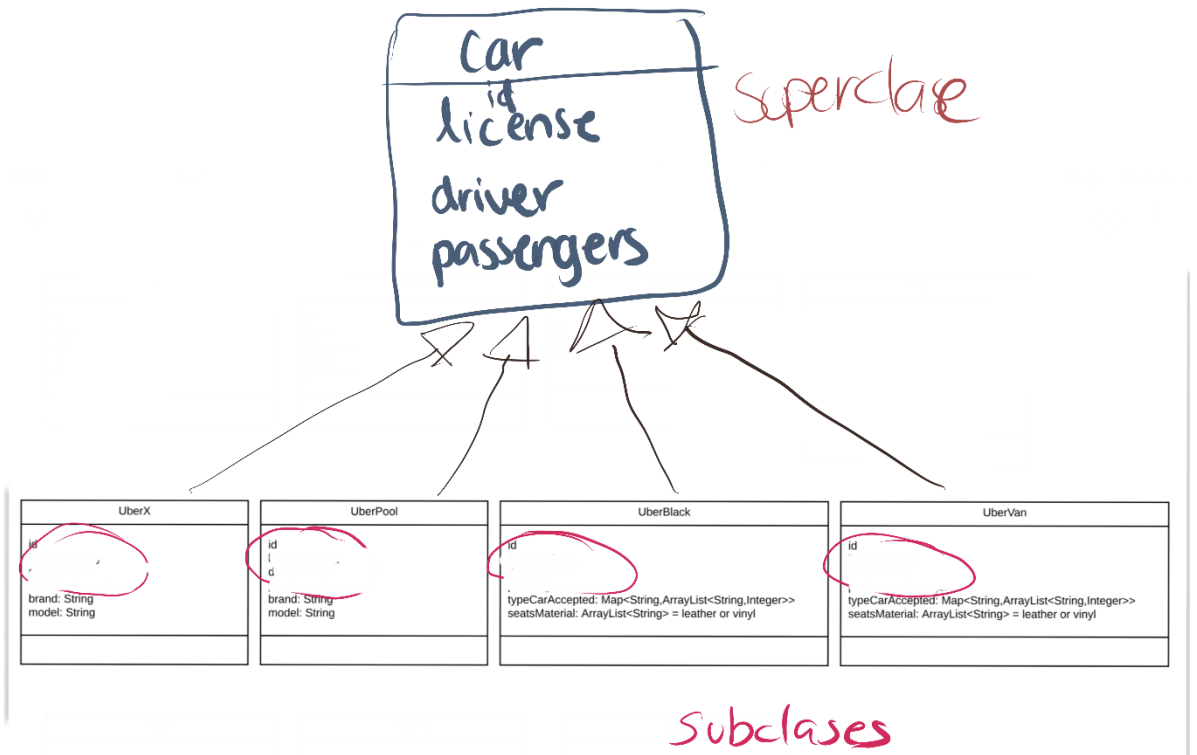




Platzi


Payments
id
Superclase
Card
id
number: Integer
cvv: Integer
date: String
PayPal
id
email: String
Cash
id
subclases


Account
id
name
document
email
password
Superclase
padre
User
Driver
subclases - hijas


Car
id
license
driver
passengers
superclase
UberX
id
brand: String
model: String
UberPool
id
brand: String
model: String
UberBlack
id
typeCarAccepted: Map<String,ArrayList<String,Integer>>
seatsMaterial: ArrayList<String> = leather or vinyl
UberVan
id
typeCarAccepted: Map<String,ArrayList<String,Integer>>
seatsMaterial: ArrayList<String> = leather or vinyl
Subclases

Objetos, método constructor y su sintaxis en código

Java	Python
Person person = new Person();	persona = Person()
JavaScript	**PHP**
var person = new Person();	$person = new Person();

Los objetos nos ayudan a crear instancias de la clase, un objeto es una instancia de la clase, el objeto es el resultado de lo que se modelo.

Los paréntesis en programación siempre van a representar métodos o acciones .

Person juan=new Person ();

Person: El tipo de dato que será nuestro objeto persona (juan)

juan: el nombre del objeto del cual se crea una instancia de la clase (En este caso la clase Person)

=: Esto es para cargarlo en memoria. Si declaramos una instancia de Person llamada juan, pero sin ponerle lo que sigue después del igual esto no se guarda en memoria

new: se llama al constructor

Person (): Usamos el constructor con las variables que le debemos asignar a la instancia del objeto.

; El punto y coma significa el final de esta instrucción

Método Constructor

Método constructor

- Dar un estado inicial al objeto
- Tiene el mismo nombre de la clase
- Son los parámetros mínimos que necesita el objeto para que pueda vivir

Person();

Objetos, método constructor y su sintaxis en código

Los objetos nos ayudan a crear instancia de una clase, el objeto es el resultado de lo que modelamos, de los parámetros declarados y usaremos los objetos para que nuestras clases cobren vida.

Los métodos constructores dan un estado inicial al objeto y podemos añadirle algunos datos al objeto mediante estos métodos. Los atributos o elementos que pasemos a través del constructor serán los datos mínimos que necesita el objeto para que pueda vivir.

Lo anteriormente hecho solo fueron clases, no pudimos nada en pantalla más que unos mensajes que decían "hola mundo", pero sin poder hacer verdaderamente nada. Para poder utilizar los elementos declarados dentro de esas clases empezaremos a trabajar con los objetos.

Recordemos que los **objetos** nos ayudaran a crear instancias de una clase, es decir, es el resultado de lo que moldeamos, de los parámetros declarados y usaremos los objetos para que nuestras clases cobren vida.

Declarar objetos

Java
```
Person person = new Person();
```

Python
```
persona = Person()
```

JavaScript
```
var person = new Person();
```

PHP
```
$person = new Person();
```

- **Java:** Al momento de crear objetos en Java, debemos tener claras dos cosas indispensables, la primera es el nombre de la clase para la cual vamos a crear el objeto y segundo el constructor que dicha clase posee, es decir, si el constructor recibe o no parámetros. Para crear objetos en Java, el lenguaje nos proporciona el comando ***new***, con este comando le decimos a Java que vamos a crear un nuevo objeto de una clase en específico y le enviamos los parámetros (en caso de ser necesario) según el constructor.
  NombreClase miObjeto = new NombreClase();

- **Python:** Haciendo valer su fama como un lenguaje flexible, para declarar un objeto es bastante sencillo ya que solo necesita el nombre del objeto y la clase a la que hara instancia.
  miObjeto = nombreClase()

- JavaScript: Al igual que con muchas cosas en JavaScript, la creación de un objeto a menudo comienza con la definición e iniciación de una variable. También hacemos uso del comando ***new*** para crear un nuevo objeto de una clase específica.
  var miObjeto = new NombreClase();

- PHP: Para crear una instancia de una clase, se debe emplear la palabra reservada new. Un objeto se creará siempre a menos que el objeto tenga un constructor que arroje una excepción en caso de error. Las clases deberían ser definidas antes de la instanciación (y en algunos casos esto es un requerimiento).
  $miObjeto = new NombreClase();

Platzi

Método constructor
Un constructor es un método especial de una clase que se llama automáticamente siempre que se declara un objeto de esa clase. Su función es inicializar el objeto y sirve para asegurarnos que los objetos siempre contengan valores válidos.

Para nosotros, las paréntesis representaran los métodos.

Java

```
public Person(String name){
   this.name = name;
}
```

Python

```
def __init__(self, name):
   self.name = name
```

JavaScript

```
function Person(name) {
   this.name = name
}
```

PHP

```
public function __construct($name){
   $this->name = name;
}
```

- **Java:** En el lenguaje Java, si para una clase no se define ningún método constructor se crea uno automáticamente por defecto. El constructor por defecto es un constructor sin parámetros que no hace nada. Los atributos del objeto son iniciados con los valores predeterminados por el sistema.

```
public nombreClase(String parámetro) {
    this.variable = parámetro;
}
```

- **Python:** En Python, el método constructor siempre se llama __init__ (dos subrayados antes y después de init).

```
def __init__(self, parámetro):
    self.variable = parámetro
```

- **JavaScript:** En JavaScript, la función sirve como el constructor del objeto, por lo tanto, no hay necesidad de definir explícitamente un método constructor. Cada acción declarada en la clase es ejecutada en el momento de la creación de la instancia.

```
function nombreClase(parámetro) {
    this.variable = parámetro;
}
```

- **PHP**: Debemos definir un método llamado __construct (es decir utilizamos dos caracteres de subrayado y la palabra construct). El constructor debe ser un método público (public function).

```
public function __construct($parámetro) {
    this->variable = parámetro;
}
```

NOTA: No te preocupes por entender this, no te compliques, lo estaremos viendo más adelante.

Platzi

Pasar datos por parámetros

Dependiendo del tipo de dato que enviemos a la función, podemos diferenciar dos comportamientos: **Paso por valor** (se crea una copia local de la variable dentro de la función) y **Paso por referencia** (se maneja directamente la variable, los cambios realizados dentro de la función le afectarán también fuera).

Java

```
Person person = new Person("Ann");
```

Python

```
person = Person("Ann")
```

JavaScript

```
var person = new Person("Ann");
```

PHP

```
$person = new Person("Ann");
```

- **Java**

  NombreClase miObjeto = new NombreClase("Ann");

  Python

  MiObjeto = NombreClase("Ann")

  JavaScript

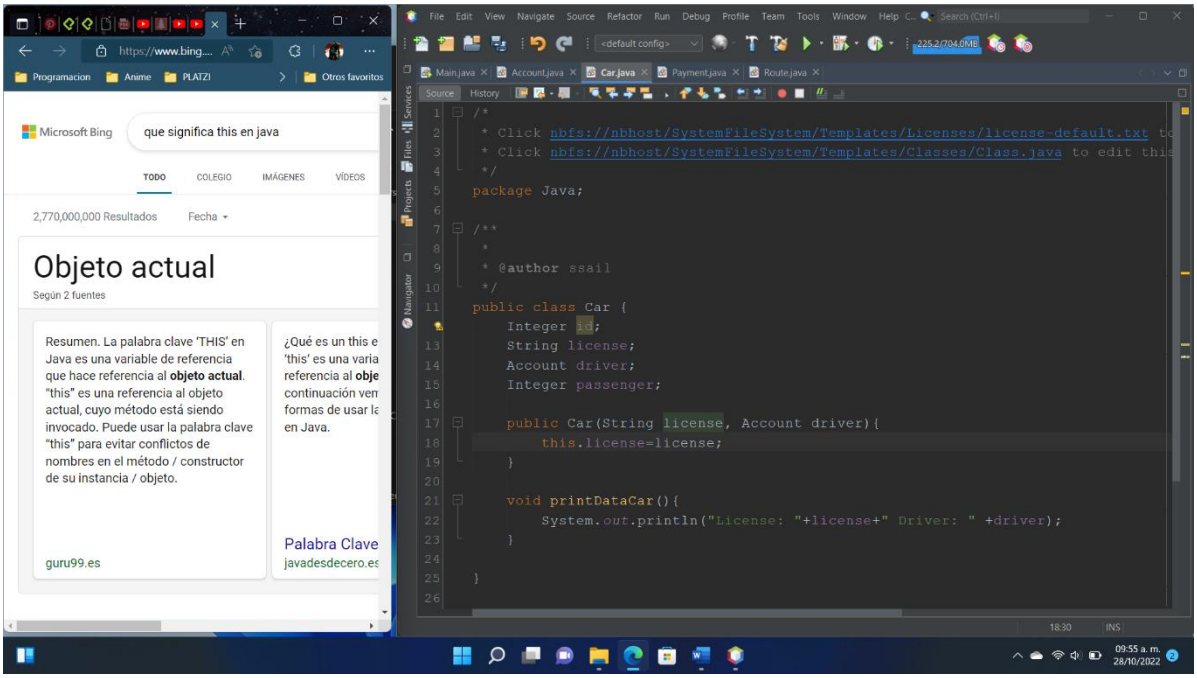
  var MiObjeto = new NombreClase("Ann")

  PHP

  $MiObjeto = new NombreClase("Ann");

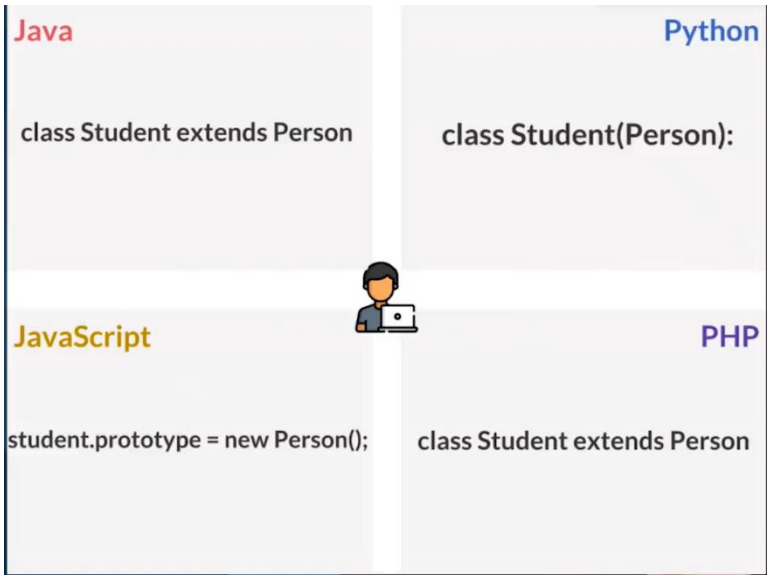
Como es visible en todos los casos, para enviar un valor es únicamente necesario ponerlos dentro de los paréntesis. Como en este caso estamos enviando un cadena de caracteres o string, ponemos comillas dobles, algo que no es necesario cuando enviamos como valor un número.

Investiga que es this, ArrayList, Map, super, Métodos constructores.



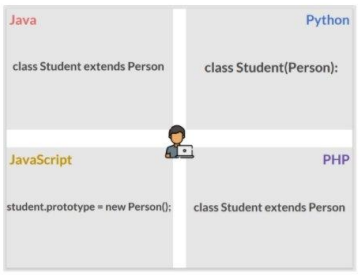
Estoy haciendo la practica ahorita sacamos apunte xd…

Aplicando herencia en código



Tema: Aplicando herencia en lenguaje Java y PHP

Puntos Clave



Notas

En Java ☕

- Si no se incluye uno, el compilador genera un constructor por defecto.
- `super` nos permite invocar los **métodos de la clase padre** y acceder a su **constructor** .
  - Esto **elimina la confusión** entre superclases y subclases que tienen **métodos con el mismo nombre**.

En php 🐘

- Para acceder a los métodos y sobretodo al constructor usamos la palabra clase `parent` .

```java
package Java;

public class UberX extends Car{
    String brand;
    String model;
    public UberX(String license, Account driver, String brand, String model){
        super(license, driver);
        this.brand = brand;
        this.model = model;
    }
}
```

```php
<?php
    class UberX extends Car
    {
        public $brand;
        public $model;

        public function __construct ($license, $driver, $brand, $model){
            parent::__construct($license, $driver); //clase padre se inte
            $this->brand = $brand;
            $this->model = $model;
        }
    }
?>
```

📌 RESUMEN: Cada lenguaje tiene su propia sintaxis para definir la herencia y por ende, sus propias palabras reservadas; *aunque como la lógica es la misma, vamos a ver que no dejan de ser bastante similares* (`super` vs `parent`)

Platzi

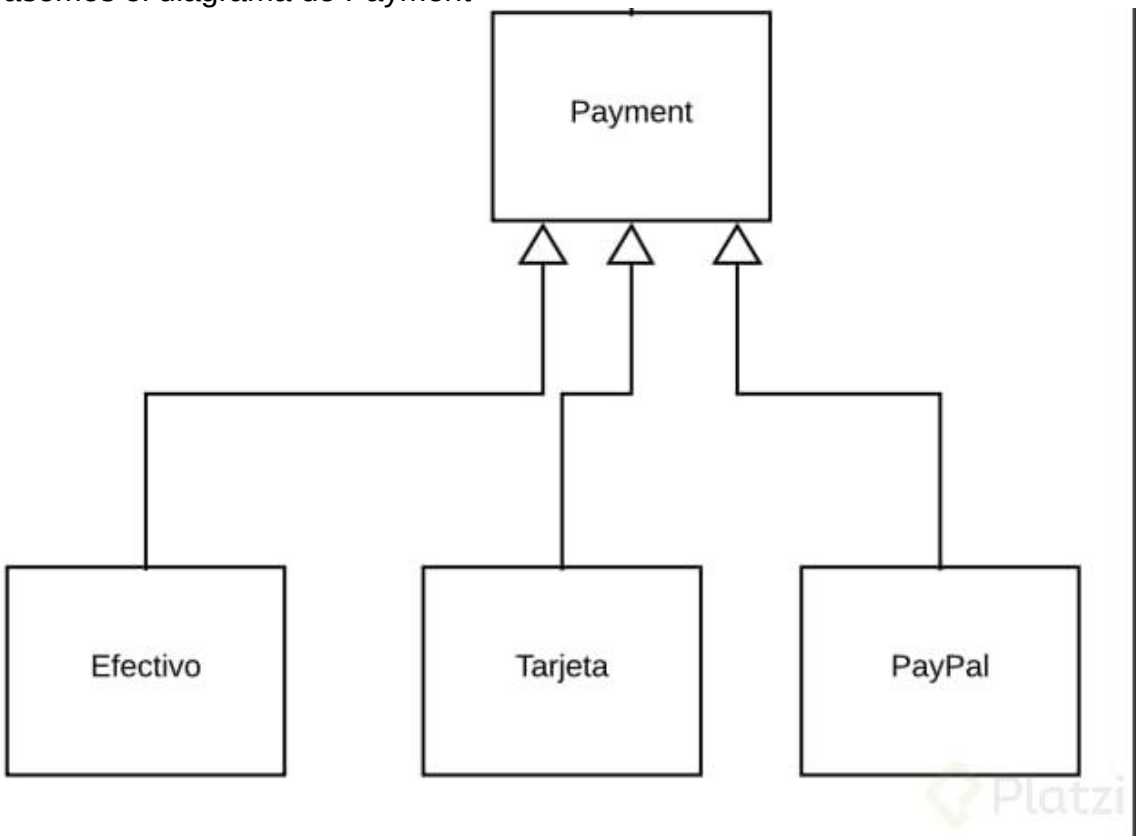
Otros tipos de herencia (Payment)

A partir de ahora las clases que estén siendo heredades las llamaremos familias.

Acabamos de aplicar herencia a la familia Car. Ahora apliquémosla a la familia Payment.

En clases anteriores te mencioné que otro punto de partida que puedes tomar para aplicar herencia es del hecho de que hay clases que lógicamente deberían estar en una familia, como es el caso de Payment.

Repasemos el diagrama de Payment



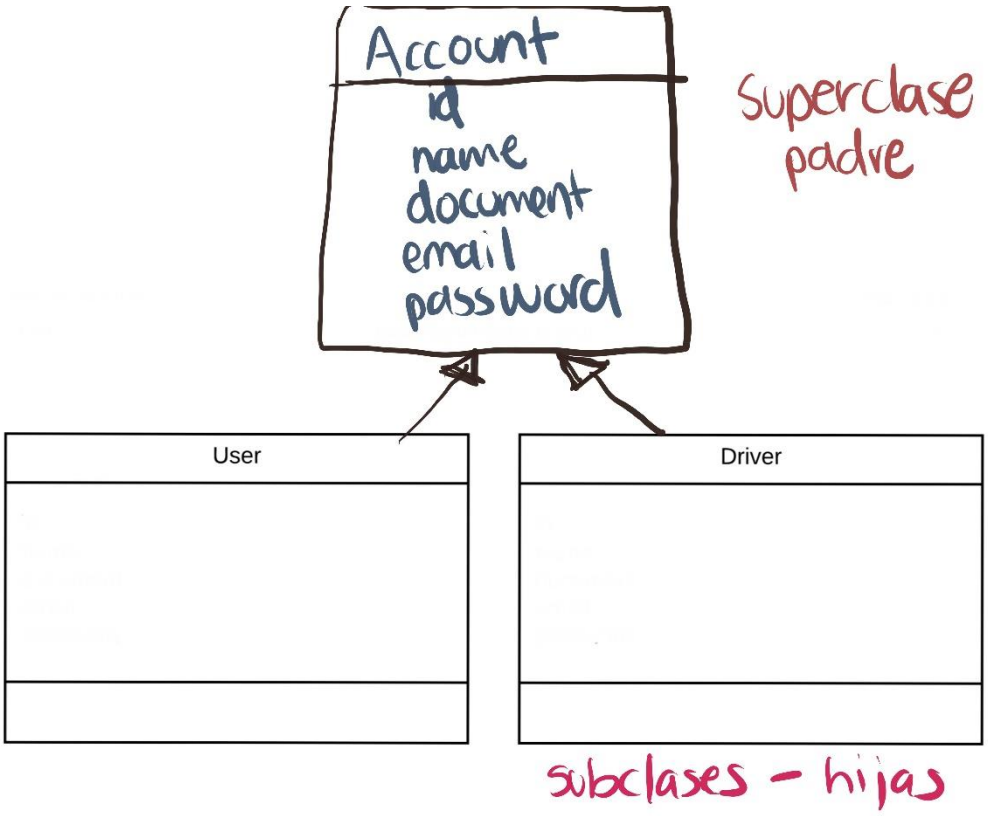
Notarás que a nivel de código parece inservible pero cuando estemos en el caso de uso Pagar un Viaje, probablemente en ese momento no sabremos cuál es el método de pago, y necesitemos ingresar un dato lo suficientemente genérico que conceptualmente nos dé la información que necesitamos, en este caso que es un Payment. Este es un tipo de Polimorfismo y uno de los principios SOLID del software que obedece a la Inyección de Dependencias. Lo veremos más adelante a detalle.

Ahora nos faltará crear las clases y aplicar su herencia.

Account

Nos queda la Jerarquía Account pendiente.

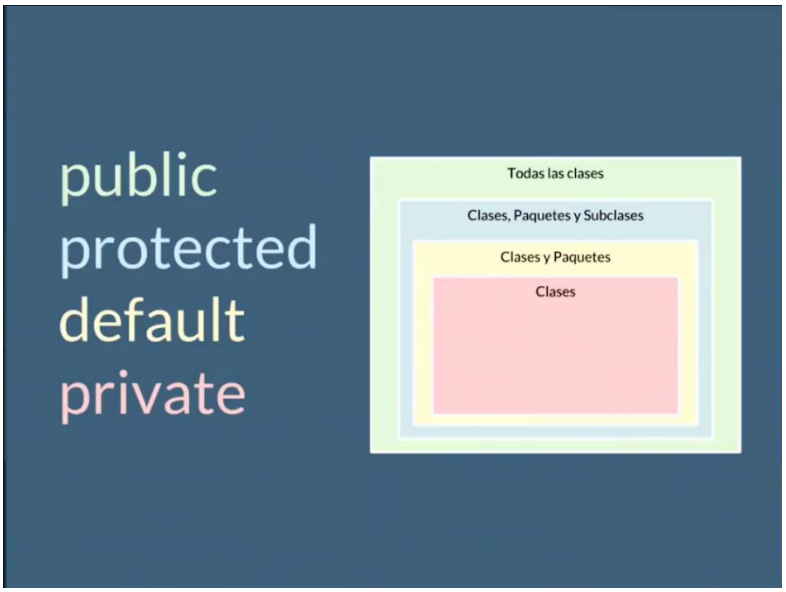
Tomando como referencia nuestros diagramas. Plásmala en tu lenguaje de programación favorito.
Compártenos tus resultados.



Platzi

Encapsulamiento

Para que un dato permanezca inviolable, inalterable, se le asigna un modificador de acceso. Para poder esconder datos se utilizan los modificadores de acceso.



Getter y Setters

Investiga, no se que pedo xd, me hago una idea xdxddd

Los Getters y Setteres permiten conjugar validaciones para restringir o dar accesos a datos de elementos encapsulados.

Generando Polimorfismo en JAVA

Poli “muchas”, morfismo “formas”

Polimorfismo

- Muchas formas
- Construir métodos con el mismo nombre pero con comportamiento diferente

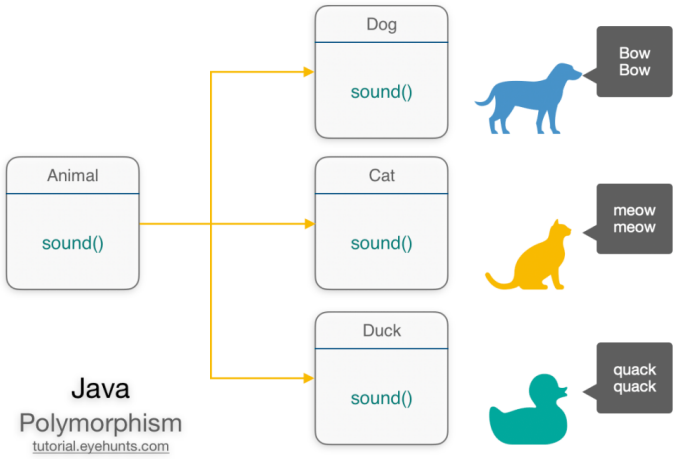
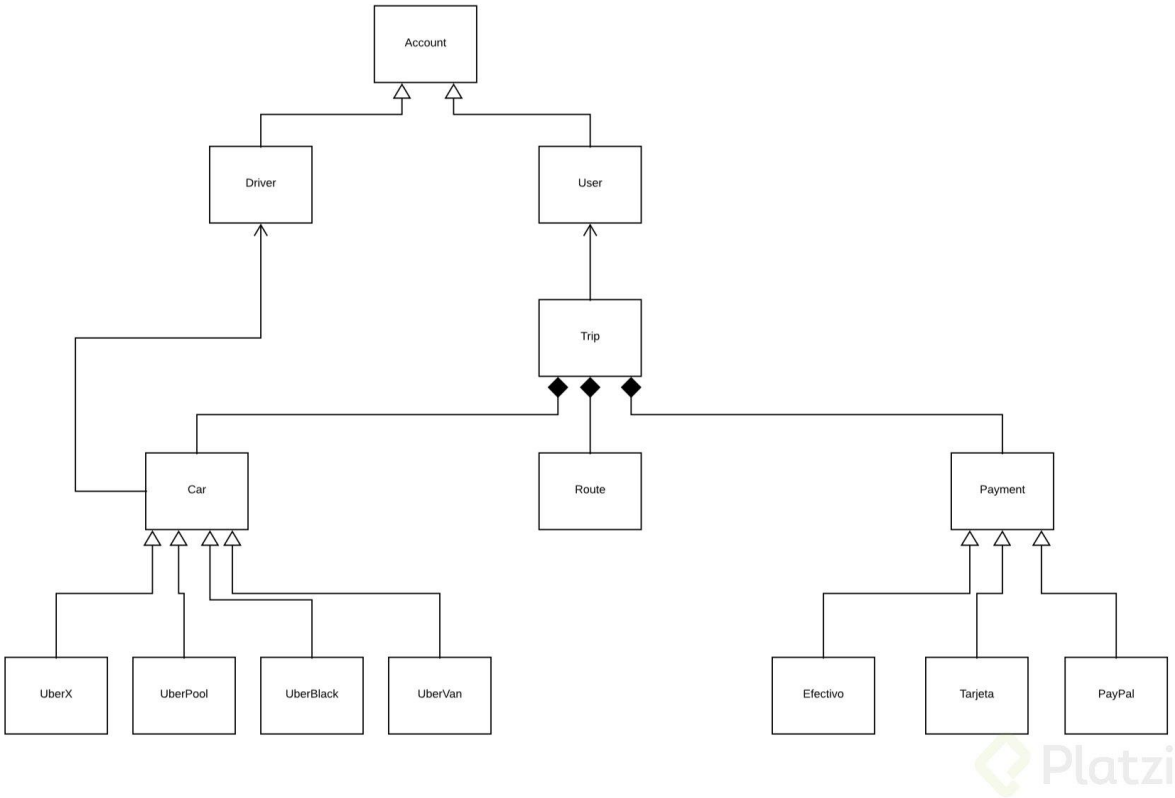


Diagrama UML de Uber

Este es el diagrama que finalmente obtuvimos, aquí solo faltaría añadirle los atributos que posee cada clase.



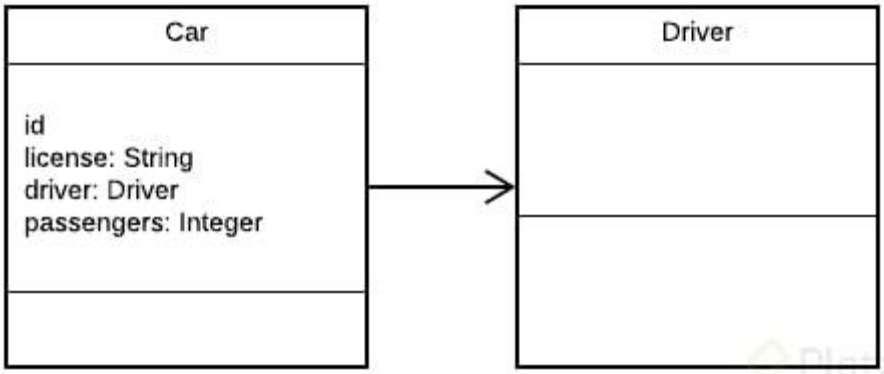
Recopilemos todo lo que hemos aprendido para explicar los últimos detalles.

En primer lugar notarás que tenemos 3 tipos de flechas:

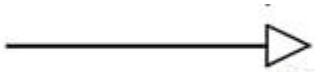
Asociación



Como su nombre lo dice, notarás que cada vez que esté referenciada este tipo de flecha significará que ese elemento contiene al otro en su definición. Si recuerdas la clase Car, este contenía una instancia de Driver. La flecha apuntará hacia la dependencia.

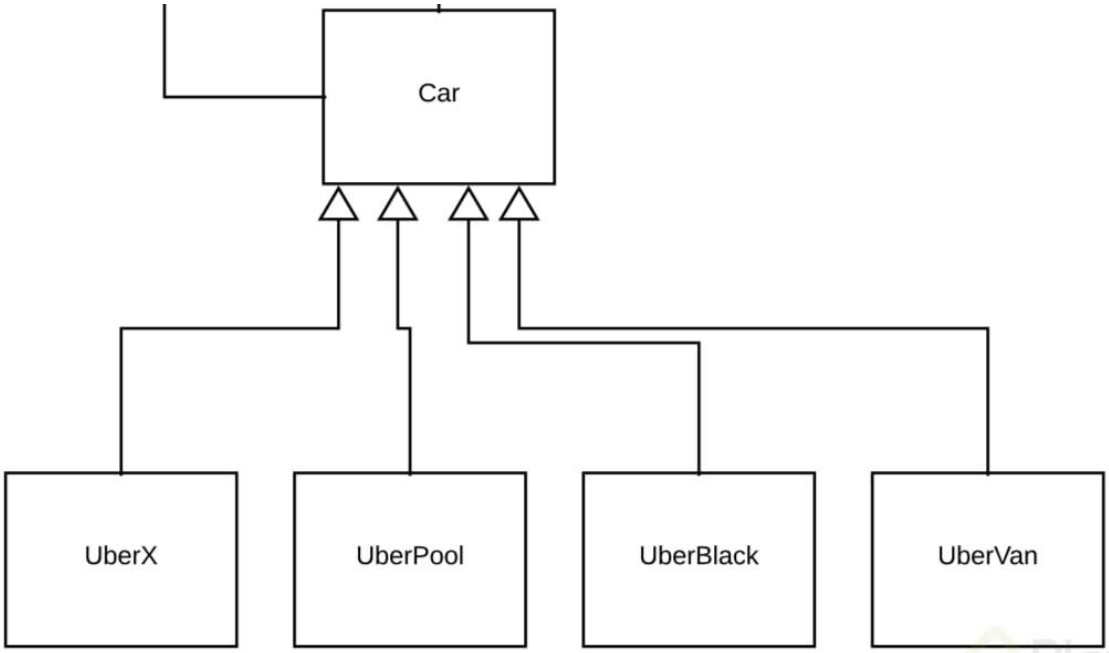


Herencia

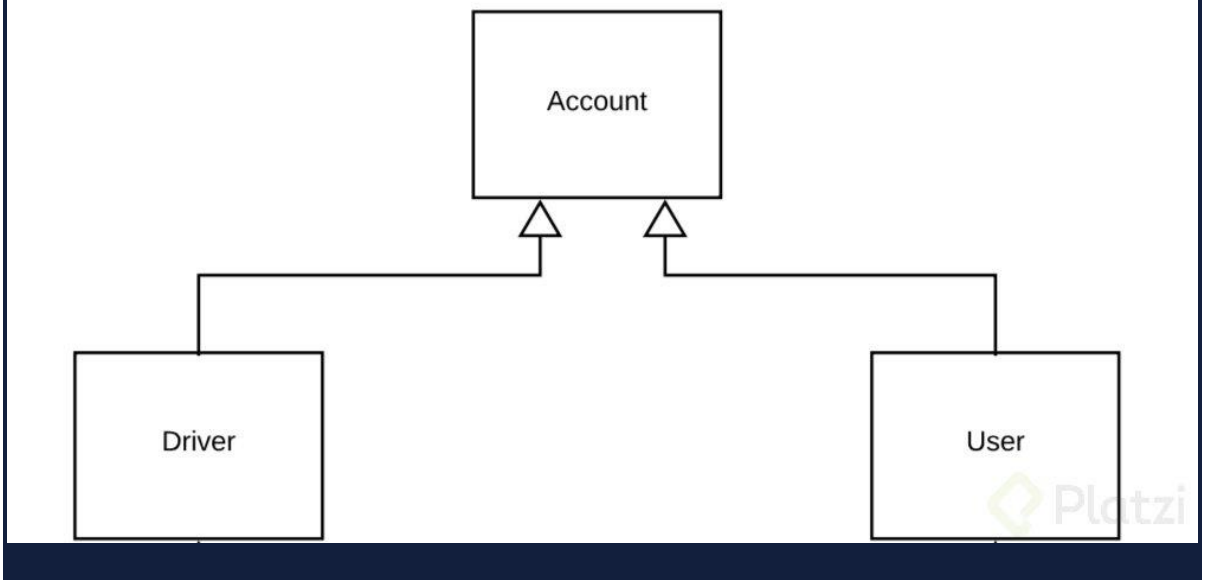


Siempre que veamos este tipo de flecha se estará expresando la herencia. En nuestro diagrama tuvimos al menos tres familias conviviendo. La dirección de la flecha irá desde el hijo hasta el padre.

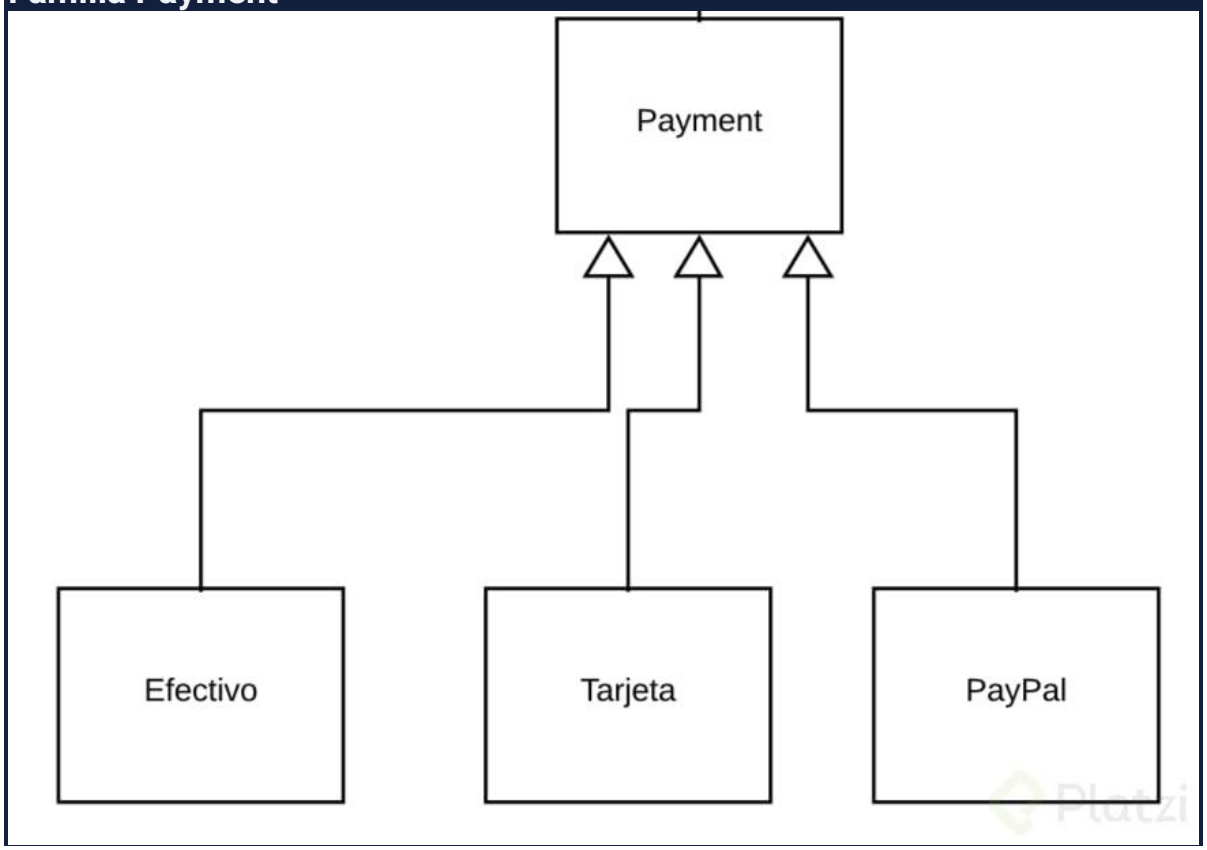
Familia Car



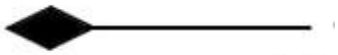
Familia Account



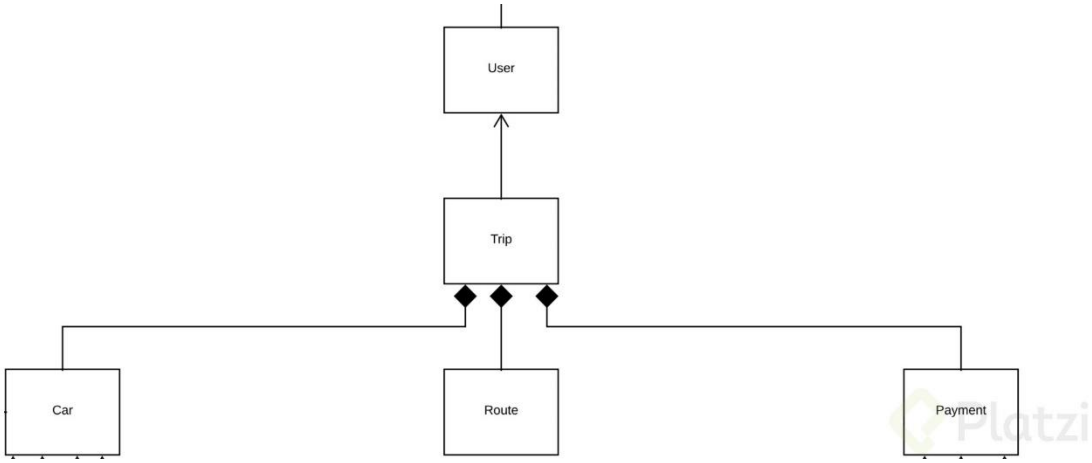
Familia Payment



Composición



Pasemos a una de nuestras piezas claves, pues notarás en el centro del diagrama la clase **Trip** que está vinculada a User, Car, Route y Payment. La composición va a significar una asociación entre estas clases con la diferencia de que para que esta clase pueda vivir forzosamente necesita a las demás. Es decir que estas clases son obligatorias para que la clase Trip pueda existir, esta dependencia obligatoria podríamos expresarla en el método constructor de la clase Trip, pues para que un objeto pueda ser creado dependerá de que los demás existan.



Esta clase Trip poseerá la lógica más fuerte del negocio aquí será donde se concentrarán la mayor cantidad de clases.

Esto es todo nuestro diagrama de clases, que quedó totalmente expresado en nuestro proyecto.